

A Safety Profile Verifying Initialization

Document #: P3402R2
Date: 2025-01-13
Project: Programming Language C++
Audience: SG23
Reply-to: Marc-André Laverdière, Black Duck Software
<marc-andre.laverdiere@blackduck.com>
Christopher Lapkowski, Black Duck Software
<redacted@blackduck.com>
Charles-Henri Gros, Black Duck Software
<redacted@blackduck.com>

Contents

- 1 Abstract
- 2 Introduction
 - 2.1 Industry Demand
- 3 Design Objectives
 - 3.1 Main Objective
 - 3.2 Profile Quality Objectives
 - 3.3 Non-objectives
- 4 Definitions
- 5 Exemptions
- 6 Interactions with Non-Verified Code
- 7 Constraints on Code
 - 7.1 General Constraints
 - 7.2 Verified Globals
 - 7.3 Verified Classes
 - 7.4 Verified Functions
 - 7.4.1 For All Verified Functions
 - 7.4.2 For Constructors of Verified Classes
- 8 Templates

- [9 Discussion](#)
 - [9.1 Alternative to `std::verified\_cast`](#)
 - [9.2 Necessity of `no.reassign?`](#)
 - [9.3 Non-Constructor Delegating](#)
 - [9.4 Divergences from \[P3274R0\]](#)
 - [9.5 Overlap with \[P3081R1\]](#)
 - [9.6 Overlap with \[P1179R1\]](#)
 - [9.7 Risks of Inconsistency](#)
- [10 Future Work](#)
- [11 Conclusion](#)
- [12 Straw Polls Concluded](#)
- [13 Straw Polls To Consider](#)
- [14 Revision History](#)
- [15 References](#)

§ 1 Abstract

We propose an attribute that specifies that ensures initialization of variables to determinate values, under a limited set of assumptions. This profile’s sole objective is to prevent undefined or erroneous behavior related to a lack of initialization. This safety profile prohibits some C++ features, and restricts constructors. Existing code bases are likely to violate these constraints, and thus this feature is an opt-in.

§ 2 Introduction

There is a growing push towards greater memory safety and memory-safe languages. While C++ is not memory-safe, it is desirable to specify and opt-in mechanism allowing a subset of C++ features that would result in memory safe programs. This has been termed ‘profiles’ ([P3274R0]), and would be specified at the TU level using an attribute.

In this paper, we propose the initialization profile, which operates at the ‘enforce’ level ([P3081R1]), and provides guarantees about variables’ initialization.

The examples in this paper assume that the profile is enabled at the ‘enforce’ level, unless annotated otherwise.

Example:

```
struct parent1 {  
    int i;
```

```

parent1() = default; //profile-rejected:, i is default initialized
};
struct child1 : public parent {
    int j;
    child1() : parent1(), j(42) {} //child is compliant, but parent isn't
}

```

§ 2.1 Industry Demand

Industry compliance standards, such as CERT C++ [CERT], forbid access of uninitialized memory (Rule EXP53-CPP). While they imply complete initialization, they do not specify how a good constructor would achieve that objective.

However, the automobile safety industry desires fully initialized class objects. As part of the The MISRA C++ standard [MISRA], there are two rules that specifically advise proper initialization of class objects.

MISRA C++2023 Rule 15.1.2 “All constructors of class should explicitly initialize all of its virtual base classes and immediate base classes”

MISRA C++2023 Rule 15.1.4 “All direct, non-static data members of a class should be initialized before the class object is accessible”

§ 3 Design Objectives

§ 3.1 Main Objective

The main objective of is profile is to eliminate the risk of undefined/erroneous behavior due to uninitialized memory, with the following assumptions:

- Inputs to verified code are initialized to determinate values
- There are no dangling pointers or references (i.e. the `std::lifetime` profile is enabled)
- There is no `constexpr` stripping (i.e. the `std::type` profile is enabled)
- The programmer uses the exemption mechanisms in a manner that’s memory-safe.

§ 3.2 Profile Quality Objectives

In addition, we have following non-functional objectives:

Simplicity of specification and use:

- the set of rules should be small, with few exceptions.
- minimal or no annotations

Simplicity of verification:

- Verification should be done by intraprocedural analysis.
- Verification should not require dataflow analysis.

Industrial applicability

- This profile should be useable on older code bases in an incremental manner.
- The profile should rely on C++11 language features, except for any attribute specified in a later version of C++.
- Verified code should be able to interact with non-verified code in a safe manner.

§ 3.3 Non-objectives

This profile does not address the handling pointer and overrun problems.

§ 4 Definitions

A *verified global* is a variable with static storage duration (6.7.6.2 [basic.stc.static]) or thread storage duration (6.7.6.3 [basic.stc.thread]) which are affected by the profile attribute that is not exempted from verification.

A *verified class* is a class that is affected by the profile attribute.

A *verified function* is a function affected by the profile attribute. This includes the member functions of a verified class, and lambdas defined by a verified function.

A *verified data member* is a non-static data member of a verified class that is not exempted from verification.

A *verified variable* is a verified global, a verified data member, a variable with automatic storage duration in a verified function, or a verified function's formal parameters.

An *object parameter* is either the `this` pointer or an explicit object parameter (9.3.4.6 [dcl.fct]).

Acceptable inputs are:

1. The non-exempt transitive closure of reachable verified variables
2. Manifestly constant-evaluated expressions (7.7 [expr.const])
3. Object Parameters
4. Arithmetic operations whose operands are the above (except pointer arithmetic)
5. Return values of verified functions when the arguments of the function call are all acceptable inputs.
6. Return values of `std::verified_cast`.

The *non-exempt transitive closure* of X means the set of symbols that are reachable from X using built-in the dot and arrow operators, and which include any symbol exempt from verification

For instance:

```
struct S {
    int b;
    OtherStruct* c;
    int * exempted [[indeterminate]];
    int * non_exempted;
    OtherStruct exempted_struct [[indeterminate]];
};

void verified_function(S s) {
    auto a = s; //acceptable
    auto b = s.b; //acceptable
    auto c = s->c->c1.c2(); //acceptable if c2 is a
        verified function
    auto d = s->exempted; //not acceptable
    auto e = s->non_exempted[32]; //not acceptable
    auto f = verified_cast(s->non_exempted[32]); //acceptable
    auto g = *s->non_exempted; //not acceptable
    auto h = &s.b; //not acceptable
    auto i = s.exempted_struct.c1; //not acceptable
}
```

Note: The use of built-in operators (e.g. `*v`, `&v`, `v[]`) to acceptable inputs is not allowed, unless it is used within `std::verified_cast()`. While we can ensure that a memory region is initialized, the program could still overrun the buffer.

§ 5 Exemptions

Developers could exempt variables from verification using the `[[indeterminate]]` attribute from [P2795R5].

```
struct HighPerformance {
    std::byte* buf [[indeterminate]];
    int sz = -1;
    void fill(/*...*/);
};
```

§ 6 Interactions with Non-Verified Code

Verified variables can be passed to non-verified functions by copy. In addition, verified variables can be initialized or assigned with non-verified data through `std::verified_cast`. This is similar to the Rust `unsafe` expression.

```
[[profiles::suppress(std::initialization)]]
int non_verified(int a);
```

```
int verified_var1 [[profiles::suppress(std::initialization)]] = /**/;
int verified_var2 = std::verified_cast(non_verified(verified_var1));
```

§ 7 Constraints on Code

The following constraints must be satisfied by all code under the purview of that profile, except

- Program constructs that are argument to `std::verified_cast`
- Program constructs not subject to being profile-rejected (as mentioned in `[dcl.attr.profile]` in [P3081R1](#)).
- Program constructs subject to being profile-rejected that are not potentially-evaluated (7.2.3 [\[expr.context\]](#)).

Otherwise, the translation unit is profile-rejected.

§ 7.1 General Constraints

For all verified variables the following constraints apply:

- `general.always.init` Default initialization that result in no initialization (9.4 [\[dcl.init\]](#)) is prohibited. This rule also applies to arrays and dynamically-allocated arrays (7.6.2.8 [\[expr.new\]](#)).
- `general.verif.init` Verified variables can only be initialized by acceptable inputs. Likewise, when verified variables are the target of assignments, the assigned value must be a verified input.
- `general.type` Verified variables' type can only be PODs and verified classes, or pointers or references thereof.

Examples:

```
struct pod {
    int i;
    int j;
};
struct DefaultDoesNotInitialize {
    pod p;
    DefaultDoesNotInitialize() = default;    //profile-rejected:
                                              general.always.init
};

pod podFactory() {
    pod p;    // profile-rejected: general.always.init
    return p;
}

[[profiles::suppress(std::initialization)]]
void non_verified_function();
struct InitsWithNonVerified {
```

```

int _i;
int _j;
    InitsWithNonVerified(int &i) : _i(i), _j(non_verified_function())
        //profile-rejected: general.verif.init
    {}
};

void UnsafeUpdateArg(pod& p) {
    p.i = non_verified_function(); //profile-rejected: general.verif.init
}

class [[profiles::suppress(std::initialization)]] UnverifiedClass { /**/
};

void non_verified_in(const UnverifiedClass &uc) { //profile-rejected:
    general.type
    //...
}

```

§ 7.2 Verified Globals

Variables with either static storage duration (6.7.6.2 [\[basic.stc.static\]](#) - including static data members (11.4.9.3 [\[class.static.data\]](#)) in a verified class) or thread storage duration (6.7.6.3 [\[basic.stc.thread\]](#)) are guaranteed to be initialized with constant initialization (6.9.3.2 [\[basic.start.static\]](#)). However, they can be reassigned with dynamic initialization (6.9.3.3 [\[basic.start.dynamic\]](#)).

Dynamic initialization can lead to subtle bugs, such as:

- Circular dependencies
- Use of statically-initialized values before they are dynamically-initialized.
- Initialization order issues when dealing with symbols variables defined in other translation units.
- Assignment of uninitialized values

We illustrate how uninitialized memory can affect static data members with dynamic initialization below.

```

struct GetsCorrupted {
    GetsCorrupted() : thefield(0) {} //compliant
    int thefield;
};

struct Wrapper {
    Wrapper() = default; //Not a POD
    static GetsCorrupted wrapped;
};

[[profiles::suppress(std::initialization)]]
GetsCorrupted corruptingFactory() {
    GetsCorrupted ret{}; //All initialized, good
}

```

```

    ret.thefield = randomInt(); //Now, some uninitialized memory snuck in
    return ret;
}
GetsCorrupted Wrapper::wrapped = corruptingFactory(); //profile-rejected:
    global.static.init, general.verif.init

```

The use of verified functions improve the picture, but initialization order issues remain:

```

extern int externInt;
int readsExtern() {
    return externInt; //compliant
}
int globalIntInitWithVerifFunc = readsExtern(); //profile-rejected:
    global.static.init

```

For all variables with static storage duration and thread storage duration affected by this profile, the following constraint applies:

- `global.static.init` Verified globals can only be initialized using constant or zero initialization.

§ 7.3 Verified Classes

All verified classes must satisfy the following property:

- `base.are.verified` All base classes of verified classes must be verified classes.

Example:

```

//Not a verified class, but would be compliant if it were
struct [[profiles::suppress(std::initialization)]] NotVerifiedBaseClass {
    int i = 0;
    NotVerifiedBaseClass() = default;
};
struct VerifiedDerivedClass : public NotVerifiedBaseClass {
    int j;
    VerifiedDerivedClass() : NotVerifiedBaseClass(), j(42) {} //profile-
        rejected: base.are.verified
};

```

§ 7.4 Verified Functions

§ 7.4.1 For All Verified Functions

All verified functions must also satisfy the following properties:

- `restrict.returns` The function can only return an acceptable input, except for a reference or a pointer to a local variable.
- `no.ref.args` Acceptable inputs can be passed to functions as follows:
 - Verified functions: copy, reference, pointer
 - Non-verified functions: copy, const reference, const pointer

Note: `restrict`.`returns` implies that lambdas defined in a verified function cannot be returned.

Examples:

```
void SafeUpdateArg(pod& p) {
    p.i = verified_function(); //Compliant
}

int verified_uses_unverified_compliant(int i) {
    int tmp = verified_cast(non_verified_function()); //Compliant
    return i * tmp;
}

[[profiles::suppress(std::initialization)]]
void non_verified_function(int& mutate);

struct CallsNonVerifiedWithReference {
    int _i;
    int _j;
    CallsNonVerifiedWithReference(int &i) : _i(i), _j{} {
        non_verified_function(i); //profile-rejected: no.ref.args
    }
};
```

§ 7.4.2 For Constructors of Verified Classes

In addition to the properties that apply to verified functions, all constructors of a verified class must satisfy the following properties:

- `init.before.read` All verified data members must be initialized before being read.
- `init.all` Except for delegating constructors, the constructor must initialize all verified data members.
- `init.list` When verified data members must be initialized, the constructor must use the *mem-initializer-list* (11.9.3 [[class.base.init](#)]) to initialize them.
- `no.reassign` Constructor bodies may not assign to verified data members.

Note regarding `init.list`: The following data members are exempt from the `init.list` criteria:

- is declared with a *brace-or-equal-initializer* (11.4.1 [[class.mem.general](#)])
- is of a type with a default constructor and its omission from the *mem-initializer-list* would result in the use of that default constructor

A data member is considered read whenever it is present in the function, except when:

- It is covered by the exceptions covered above.
- It is the left operand of a simple assignment (7.6.19 [[expr.ass](#)]).

Note: A nonconforming constructor would bring the rejection of its class only, and not of its subclasses. This decision is intended to reduce the noisiness that would come from a faulty constructor at the top of a very large class hierarchy.

§ 7.4.2.1 Compliant Examples

```
struct ValueInitialized {
    pod p{};
    ValueInitialized() = default; //compliant
};

struct InitWithVerifiedReturnValue {
    static pod podFactory();
    InitWithVerifiedReturnValue() : p(podFactory()) {} //compliant
};

struct WithExemption {
    std::byte* buf [[indeterminate]];
    size_t    buf_size;
    int i;
    WithExemption() : i(0), buf_size(0) {} //compliant: buf is not a
        verified data member
};

struct SafeDefaultInit {
    int i;
    int j;
    SafeDefaultInit() : i(123), j(456) {} //compliant
};

struct ReliesOnDefaultInit {
    int i;
    SafeDefaultInit sdi;
    ReliesOnDefaultInit() : i(123) {} //compliant: sdi has a default ctor
};

struct MixedInits {
    int i;
    int j;
    int z = 0;
    MixedInits() : i(123), j(456) {} //compliant: verified data members are
        initialized using either allowed mechanism
};

struct WithCallInCtorBody {
    int i;
    int j;
    void utility_function() const;
    WithCallInCtorBody(int i) : i(i), j() {
        utility_function(); //compliant: calling a verified function with
            'this'
    }
};

struct UpdatesGlobal {
```

```

static unsigned num_allocations;
UpdatesGlobal() {
    ++num_allocations; //compliant: a verified input is updated with the
                        result of an arithmetic operation over verified inputs
}
};
unsigned UpdatesGlobal::num_allocations = 0;

struct CallsVerifiedNonConst {
    int i;
    int j;
    void mutating();
    CallsVerifiedNonConst(int i) : i(i), j{} {
        mutating(); //compliant, but could have a redefinition of i or j
    }
};

```

§ 7.4.2.2 Profile-Rejected Examples

```

struct WrongOrder {
    int i;
    int j;
    WrongOrder() : i(j), j(42) {} //profile-rejected: init.before.read
};

```

```

struct MissingInit {
    int i;
    MissingInit() : {} //profile-rejected: init.all
};

```

```

struct InitInCtorBody {
    int i;
    int j;
    int z = 0;
    InitInCtorBody() {
        i = 123;
        j = 456;
        //profile-rejected: init.list
    }
};

```

```

struct ReassignInCtor {
    int i;
    int j;
    ReassignInCtor() : i(123), j(456) {
        j = verified_function(); // profile-rejected: no.reassign
    }
};

```

```

struct CallsNonVerifiedWithFieldReference {
    int _i;
    int _j;
    CallsNonVerifiedWithFieldReference(int &i) : _i(i), _j{} {
        non_verified_function(_j); //profile-rejected: no.ref.args
    }
};

```

§ 8 Templates

In the case of templated classes, the property is verified during template instantiation.

```
class [[profiles::suppress(std::initialization)]] Suppressed {/**/};
class Enforced {/**/};

template<typename T>
class Template {
    T field = T();
};

void foo() {
    Template<Suppressed> sup {}; //profile-rejected: calling the constructor
                                to a non-verified class
    Template<Enforced> enf {}; //compliant
}
```

§ 9 Discussion

§ 9.1 Alternative to `std::verified_cast`

In [P3081R1], the proposal is to disable a profile for a given scope (e.g. `[[profiles::suppress(...)]]`), such as a block. This would be a suitable alternative to `std::verified_cast`.

§ 9.2 Necessity of `no.reassign`?

The `no.reassign` is not strictly necessary, and might potentially prohibit legitimate use cases. This rule was suggested by [P3274R0] but we should reconsider it on the grounds of putting as few restrictions as possible in order to achieve our stated goal.

§ 9.3 Non-Constructor Delegating

A recent SG23 mailing list discussion highlighted that delegating initialization to a non-constructor member function is idiomatic in C++. Supporting this idiom would make this profile more useful.

The bit of code that triggered the discussion is the following:

```
basic_string(const _CharT* __s, const _Alloc& __a = _Alloc())
: _M_dataplus(_M_local_data(), __a)
{
    //...
    _M_construct(__s, __end, forward_iterator_tag());
}
```

In this case, `_M_local_data()` returns a `const` pointer to a data member (`_M_local_buf`) and passes it to the `_M_dataplus` data member. The initialization then is done by

`_M_construct`. This code would be reported as violating the safety profile as we specify it in this draft, since the constructor does not initialize `_M_dataplus` directly.

There are a few solutions to this problem:

1. Force developers to rewrite their code to use delegating constructors.
2. Use an on-demand interprocedural analysis that ensures that initialization happens on all paths in the callee.
3. A suggestion was to annotate arguments that must be initialized with `[[must_init]]`.

```
struct DelegatingInit {  
    int member;  
    DelegatingInit() {  
        internal_init(&member);  
    }  
    internal_init([[must_init]] int* p);  
}
```

This option is less intrusive than option 1, and would be simpler to verify than option 2, simply because the scope of the analysis becomes well-bounded. As such, it is worth considering.

Nonetheless, we consider it undesirable for the following reasons:

- Adding another attribute is undesirable.
- Only explicit parameters can be specified this way. However, developers might like to have a general `initialize()` member function.
- Functions with the `[[must_init]]` attribute can delegate further to other `[[must_init]]` functions, possibly leading to recursion.
- Verifying such an annotated function would require dataflow analysis.

Given the design objectives, we conclude that there is no viable path to accept non-constructor delegation in this profile.

§ 9.4 Divergences from [P3274R0]

This draft materially deviates from [P3274R0] in the following ways:

- We do not assume that all default constructors are initializing data members.
- We do not take position on implicit initialization and whether it is error-prone.

§ 9.5 Overlap with [P3081R1]

The rule for Type.6 proposed by [P3081R1] is identical to rule `general.always.init`. Since this rule is not related to type safety, it belongs more meaningfully to the

initialization profile.

Since this restriction is very desirable, and that [P3081R1] has a good chance of landing in C++26, we encourage [P3081R1] to specify an initialization profile containing only rule Type.6, which this paper would eventually build upon.

§ 9.6 Overlap with [P1179R1]

The rule `restrict`.returns overlaps with the lifetime safety profile to some extent ([P1179R1]). However, it is a sensible restriction to enforce. In the event that the Committee would prefer to avoid overlaps between profiles, the rule could be written as follows:

- `restrict`.returns: The function can only return an acceptable input.

§ 9.7 Risks of Inconsistency

Different TUs may have different profiles enabled. This could lead to situations where a TU mistakenly expects a symbol to comply with the requirements of this profile. Consider the example below, whereby the TU implementing `foo` does so without the initialization profile. However, another TU requires the initialization profile, and depends on `foo`.

```
//in impl.cpp
[[profiles::suppress(std::initialization)]]
int foo() { /*...*/ }

//in caller.cpp
[[profiles::enforce(std::all)]]
int foo();

int main(int argc, char** argv) {
    return foo();
}
```

There are a few possible solutions, but all of them would pose challenges to adoption.

- The profile annotation should not apply to forward declarations. In this case, TUs using this profile will have to be self-contained.
- The profile information should be added to the symbol's mangled name, so that a program like the example above would compile, but not link. Sadly, a linker error of this nature would likely be hard to troubleshoot.
- Allow this profile only to be used in a module, and require that module enforcement must be consistent between declarations and implementations when the module is built.

The option of a linker error seems to be the lesser evil for the time being.

§ 10 Future Work

We also observe that profiles imply a constraint on what types can be used in a template. This hints at a new concept. A future revision of this paper would explore this further.

§ 11 Conclusion

In this paper, we propose a safety profile that guarantees that any all code affected by the profile attribute will initialize both local and global variables to determinate values, assuming that the data used for construction is itself initialized properly. The profile does not depend on the presence of specific modern C++ features and can thus be applied to legacy code bases.

The profile introduces a single new symbol, `std::verified_cast`, which could be implemented as:

```
namespace std {  
    template<typename T>  
    T&& verified_cast(T&& i) { return std::forward(i); }  
}
```

Thus, developers on legacy code bases that are still using older versions of the C++ standard could take advantage of this profile, assuming that they define `std::verified_cast` in their code base.

§ 12 Straw Polls Concluded

The following straw polls were conducted during the Wrocław 2024 meeting.

POLL: We should promise more SG23 committee time to pursuing this paper, knowing that our time is scarce and this will leave less time for other work.

Favor	Neutral	Against
18	1	0

Strong consensus

POLL: For a given scope of applicability (eg translation unit) should this profile prohibit the use of default initialization altogether?

Favor	Neutral	Against
1	4	13

Strong consensus against

POLL: For a given scope of applicability (eg translation unit) should this profile prohibit the use of default initialization leading to no initialization?

Favor	Neutral	Against
17	0	0

Unanimous

§ 13 Straw Polls To Consider

Q1: Should an attribute be used to exempt data members from initialization?

Q2: Should specific types be used to exempt data members from initialization?

Q3: Should the profile rely on an attribute on parameters that indicates what the function is responsible for initializing?

Q4: Should the profile prohibit reinitialization in the constructor body?

Q5: What should be the mechanism to interact between the verified world and the unverified world?

Q6: Should we allow to pass verified variables by non-const reference or pointer to unverified functions? This would make the profile more useful, but offers lower guarantees.

§ 14 Revision History

R2: 2025-01-13 Initialization at large, for Hadenberg

R1: 2024-10-11 Class initialization, presented in Wrocław

R0: 2024-09-17 Early draft on class initialization for discussion with the community

§ 15 References

[CERT] SEI CERT. 2016. SEI CERT C++ Coding Standard.

<https://wiki.sei.cmu.edu/confluence/pages/viewpage.action?pageId=88046682>

[MISRA] The MISRA Consortium and Chris Tapp. 2023. MISRA C++:2023: Guidelines for the use of C++17 in critical systems.

<https://misra.org.uk/misra-cpp2023-released-including-hardcopy/>

[P1179R1] Herb Sutter. 2019-11-22. Lifetime safety: Preventing common dangling.

<https://wg21.link/p1179r1>

[P2795R5] Thomas Köppe. 2024-03-22. Erroneous behaviour for uninitialized reads.

<https://wg21.link/p2795r5>

[P3081R1] Herb Sutter. 2025-01-06. Core safety profiles for C++26.

<https://wg21.link/P3081R1>

[P3274R0] Bjarne Stroustrup. 2024-05-10. A framework for Profiles development.

<https://wg21.link/p3274r0>